In [1]:
```
import numpy as np
from datetime import datetime
import os
import pandas as pd
from os import listdir
from os.path import isfile, join
```

In [2]:
```
print("Enter post files path:")
###PostLoc = input()
PostLoc = "C:\\Users\\muhsin.caglar\\OneDrive - Retrac Productions Ltd\\Desktop\

PostLoc = PostLoc.replace("\\","\\\\")
#### C:\Users\muhsin.caglar\OneDrive - Retrac Productions Ltd\Desktop\Automation
#### C:\Users\muhsin.caglar\OneDrive - Retrac Productions Ltd\Desktop\Automation

print(PostLoc)
```

```
Enter post files path:
C:\\Users\\muhsin.caglar\\OneDrive - Retrac Productions Ltd\\Desktop\\Automation
\\PostPostProc\\2998
```

In [3]:
```
PostFiles = listdir(PostLoc)
PostFiles = [item for item in PostFiles if ".ptp" in item]
print(PostFiles)
```

```
['2998_101.ptp', '2998_102.ptp', '2998_103.ptp', '2998_104.ptp', '2998_104B.ptp',
'2998_104C.ptp', '2998_105.ptp', '2998_106.ptp', '2998_107.ptp']
```

In [4]:
```
PostFile = PostLoc + "\\\\" +  PostFiles[2]
print(PostFile)
```

```
C:\\Users\\muhsin.caglar\\OneDrive - Retrac Productions Ltd\\Desktop\\Automation
\\PostPostProc\\2998\\2998_103.ptp
```

In [5]:
```
import pandas as pd
import re

with open(PostFile, 'r') as file:
    lines = file.readlines()
#DROP LINES WITH OP NAMES ETC.
lines = [line for line in lines if "(" not in line]



data = []
#FIND ALL CODES AND CREATE COLUMNS
# Process each line
for line in lines:
    # Find all key-value pairs (e.g., "N72", "G3", "X-1083.761", etc.)
    matches = re.findall(r"([A-Z])([-+]?\d*\.?\d+)", line)

    # Convert matches to dictionary (e.g., {'N': '72', 'G': '3', 'X': '-1083.761
    line_dict = {key: value for key, value in matches}

    # Append to data list
    data.append(line_dict)

#TURN INTO DATAFRAME
df = pd.DataFrame(data)
```

```
#DROP UNWANTED COLUMNS & RESET INDEX
df = df.drop(columns=['N'])
df = df.reset_index(drop=True)
#IF G53 MACHINE MOVEMENT THAN SET Z TO 1000
df.loc[df['G'] == "53", 'Z'] = 1000

#FIND THE FIRST G69 CODE AND SET ALL X,Y,Z,I,J,K VALUES
first_69_index = df[df['G'] == "69"].index[0]
df.loc[first_69_index, ['X', 'Y', 'I', 'J', 'K']] = 0
df.loc[first_69_index, ['Z']] = 1000

#CHANGE TYPE TO FLOAT
df[['X', 'Y', 'Z', 'I', 'J', 'K']] = df[['X', 'Y', 'Z', 'I', 'J', 'K']].astype(f

#COMPUTE G91 MOVEMENTS SO THAT IT IS CUMULATIVE
for index in range(1, len(df)):
    if df.loc[index, 'G'] == "91":
        # Add the row values to the row above and replace the current row's valu
       df.loc[index, 'Z'] = df.loc[index, 'Z'] + df.loc[index - 1, 'Z']
#GET VALUES FOR 68.2 ROW. MAY NOT BE NEEDED IF WE DROP THE ROWS.
for index in range(1, len(df)):
    if df.loc[index, 'G'] == "68.2":
        # Add the row values to the row above and replace the current row's valu
        df.loc[index, 'Z'] = df.loc[index - 1, 'Z']

#DROP UNWANTED ROWS
if 'O' in df.columns:
    df = df.drop('O', axis=1)
if 'C' in df.columns:
    df = df.drop('C', axis=1)
if 'B' in df.columns:
    df = df.drop('B', axis=1)
if 'T' in df.columns:
    df = df.drop('T', axis=1)
if 'M' in df.columns:
    df = df.drop('M', axis=1)
if 'H' in df.columns:
    df = df.drop('H', axis=1)
if 'S' in df.columns:
    df = df.drop('S', axis=1)
if 'Q' in df.columns:
    df = df.drop('Q', axis=1)
if 'F' in df.columns:
    df = df.drop('F', axis=1)
if 'R' in df.columns:
    df = df.drop('R', axis=1)
if 'A' in df.columns:
    df = df.drop('A', axis=1)


############ IT ALL LOOKS GOOD UP TO HERE....
df.loc[df['G'] != "68.2", ['I', 'J', 'K']] = np.nan
df.loc[df['G'] == "69", ['I', 'J', 'K']] = 0
df[['I', 'J', 'K']] = df[['I', 'J', 'K']].fillna(method='ffill')
df = df[df['Z'] != 2000]

df = df.dropna(how='all')
values_to_remove = ["43.4", "49", "53.1", "5.1","68.2","69","53"]

# Filter out rows where column 'G' has any of the specified values
```

```
df = df[~df['G'].isin(values_to_remove)]
df[['X', 'Y', 'Z']] = df[['X', 'Y', 'Z']].fillna(method='ffill')
new_order = ['G', 'X', 'Y', 'Z', 'I', 'J','K']




# Reorder columns
df = df[new_order]
df
df.to_csv("C:\\Users\\muhsin.caglar\\OneDrive - Retrac Productions Ltd\\Desktop\
df
```

```
C:\Users\muhsin.caglar\AppData\Local\Temp\ipykernel_11820\3362880288.py:79: Futur
eWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future
version. Use obj.ffill() or obj.bfill() instead.
  df[['I', 'J', 'K']] = df[['I', 'J', 'K']].fillna(method='ffill')
C:\Users\muhsin.caglar\AppData\Local\Temp\ipykernel_11820\3362880288.py:87: Futur
eWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future
version. Use obj.ffill() or obj.bfill() instead.
  df[['X', 'Y', 'Z']] = df[['X', 'Y', 'Z']].fillna(method='ffill')
```

Out[5]:

	G	X	Y	Z	I	J	K
5	NaN	NaN	NaN	NaN	0.0	0.0	0.0
6	NaN	NaN	NaN	NaN	0.0	0.0	0.0
7	NaN	NaN	NaN	NaN	0.0	0.0	0.0
9	0	0.000	0.000	NaN	0.0	0.0	0.0
10	90	0.000	0.000	NaN	0.0	0.0	0.0
...	...	...	...	...	...	...	...
61632	NaN	-9.989	424.255	844.7	0.0	-20.0	0.0
61633	NaN	-9.989	424.255	844.7	0.0	-20.0	0.0
61639	NaN	-9.989	424.255	844.7	0.0	0.0	0.0
61640	NaN	-9.989	424.255	844.7	0.0	0.0	0.0
61641	NaN	-9.989	424.255	844.7	0.0	0.0	0.0

61587 rows × 7 columns

In [6]:
```
indices_with_90 = df.index[df['G'] == "90"].tolist()
unique_values = df["G"].unique().tolist()
indices_with_68_2 = df.index[df['G'] == "68.2"].tolist()
indices_with_69 = df.index[df['G'] == "69"].tolist()
indices_with_69
indices_with_91= df.index[df['G'] == "91"].tolist()
print(unique_values)
```

```
[nan, '0', '90', '91', '17', '1']
```

In [7]:
```
#df['G_C_Count'] = df.groupby('G').cumcount() + 1
#df['G_C_Count'] = df['G_C_Count'].fillna(0)
#df['G_C_Count'] = df['G_C_Count'].astype(int)
```

```
#df['G_C_Count'] = df['G_C_Count'].astype(str)
#df
```

In [8]:

```
import numpy as np
import pandas as pd

import numpy as np
import pandas as pd

def reverse_zxz_euler_from_df(df, x_col, y_col, z_col, i_col, j_col, k_col):
    """
    Applies the reverse ZXZ Euler transformation on coordinates in a DataFrame
    and stores the results in new columns: x_1, y_1, z_1.

    Parameters:
        df (pd.DataFrame): The input DataFrame containing the 3D points and Eule
        x_col (str): Column name for the X coordinates.
        y_col (str): Column name for the Y coordinates.
        z_col (str): Column name for the Z coordinates.
        i_col (str): Column name for the rotation angle around the first Z-axis
        j_col (str): Column name for the rotation angle around the X-axis (J).
        k_col (str): Column name for the rotation angle around the second Z-axis

    Returns:
        pd.DataFrame: A DataFrame with new columns `x_1`, `y_1`, `z_1`.
    """
    # Function to perform reverse ZXZ Euler transformation on a single row
    def transform_row(row):
        # Extract coordinates and angles
        x, y, z = row[x_col], row[y_col], row[z_col]
        i, j, k = row[i_col], row[j_col], row[k_col]

        # Convert angles to radians if they are in degrees
        i, j, k = np.radians([i, j, k])

        # Create inverse rotation matrices
        R_z2_inv = np.array([
            [np.cos(k), np.sin(k), 0],
            [-np.sin(k),  np.cos(k), 0],
            [0,           0,          1]
        ])

        R_x_inv = np.array([
            [1, 0,           0         ],
            [0, np.cos(j),  -np.sin(j)],
            [0, np.sin(j), np.cos(j)]
        ])

        R_z1_inv = np.array([
            [np.cos(i), np.sin(i), 0],
            [-np.sin(i),  np.cos(i), 0],
            [0,           0,          1]
        ])

        # Combine the inverse transformations in reverse ZXZ order
        R_inv = R_z2_inv @ R_x_inv @ R_z1_inv

        # Apply the reverse transformation to the point
        point = np.array([x, y, z])
        rev_point1 = R_z2_inv @ point
```

```
        rev_point2 = R_x_inv @ rev_point1
        reversed_point = R_z1_inv @ rev_point2

        # Return reversed coordinates
        return pd.Series(reversed_point, index=['x_1', 'y_1', 'z_1'])

    # Apply the transformation to each row of the DataFrame
    df[['x_1', 'y_1', 'z_1']] = df.apply(transform_row, axis=1)

    return df
```

In [9]:
```
#df.loc[df['G'] != "68.2", ['I', 'J', 'K']] = np.nan
#df.loc[df['G'] == "69", ['I', 'J', 'K']] = 0
```

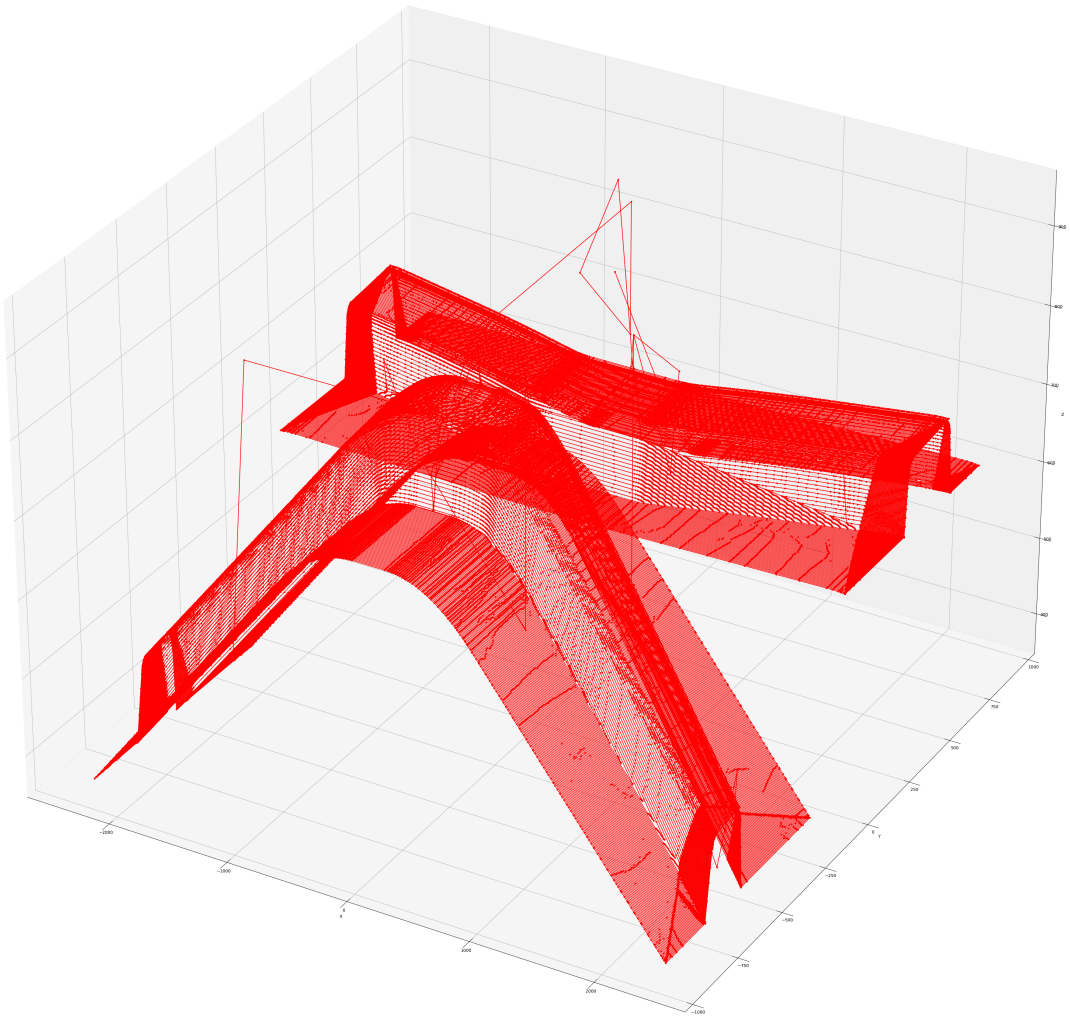
In [10]:
```
df[['I', 'J', 'K']] = df[['I', 'J', 'K']].fillna(method='ffill')
df = df[df['Z'] != 2000]
df.to_csv("C:\\Users\\muhsin.caglar\\OneDrive - Retrac Productions Ltd\\Desktop\
```

```
C:\Users\muhsin.caglar\AppData\Local\Temp\ipykernel_11820\1416041043.py:1: Future
Warning: DataFrame.fillna with 'method' is deprecated and will raise in a future
version. Use obj.ffill() or obj.bfill() instead.
  df[['I', 'J', 'K']] = df[['I', 'J', 'K']].fillna(method='ffill')
```

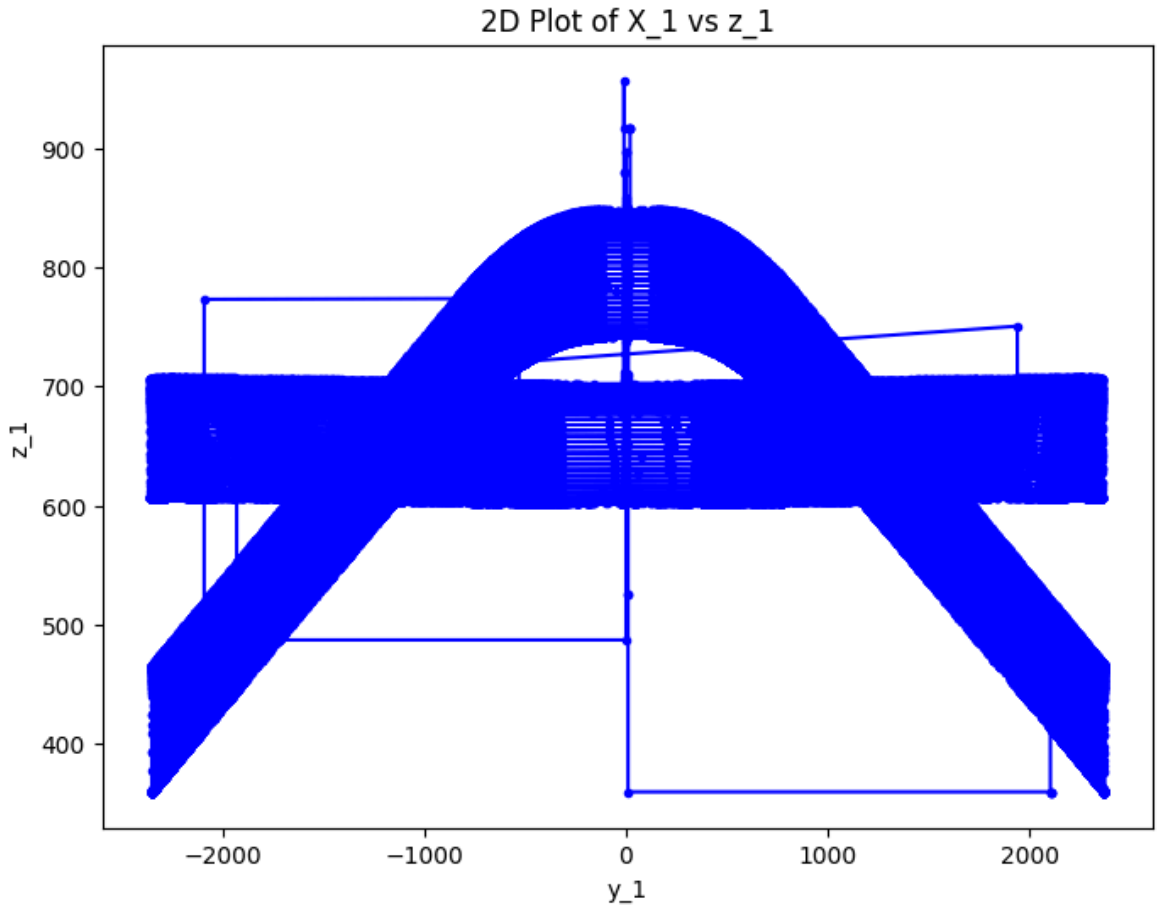
In [11]:
```
df = reverse_zxz_euler_from_df(df, 'X', 'Y', 'Z', 'I', 'J', 'K')

df.to_csv("C:\\Users\\muhsin.caglar\\OneDrive - Retrac Productions Ltd\\Desktop\
```

In [12]:
```
import matplotlib.pyplot as plt
```

In [13]:
```
fig = plt.figure(figsize=(50, 50))
ax = fig.add_subplot(111, projection='3d')


ax.plot(df['x_1'], df['y_1'], df['z_1'], color='r', label='Transformed (X_1, Y_1

# Label axes
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

# Add legend
ax.legend()

# Show plot
plt.show()
```



In [14]:
```
# Create a 2D plot of X_1 vs Y_1
plt.figure(figsize=(8, 6))
plt.plot(df['x_1'], df['z_1'], marker='.', linestyle='-', color='b')

# Add labels and title
plt.xlabel('y_1')
plt.ylabel('z_1')
plt.title('2D Plot of X_1 vs z_1')

# Show the plot
plt.show()
```



In [15]:
```
min_z1_value = df['z_1'].min()

print("Minimum value of Z_1:", min_z1_value)
```

```
Minimum value of Z_1: 359.2797478051456
```